

Practical No. 6

Aim: To develop and execute a serverless function using AWS Lambda in the AWS cloud environment.

Theory:

Serverless computing is a cloud execution model where the cloud provider dynamically manages the allocation of resources. Developers write and deploy code without provisioning or managing servers.

AWS Lambda is a serverless compute service that runs code in response to events and automatically manages the underlying infrastructure. Functions can be triggered by various AWS services such as EventBridge, S3, API Gateway, and more.

Advantages of Serverless Computing

- No server management or infrastructure provisioning required
- Highly scalable and cost-efficient for event-driven workloads
- Faster development cycles with focus on business logic

Disadvantages of Serverless Computing

- Cold start latency when a function is invoked after being idle
- Limited execution duration and resource constraints per function
- Vendor lock-in to the AWS ecosystem

Steps:

Task 1: Create a Lambda Function

1. In the AWS Console, search for and open Lambda.
2. Click Create a function. Select Author from scratch and configure: Function name as myStopinator, Runtime as Python 3.11.
3. Under Change default execution role, select Use an existing role and choose myStopinatorRole from the dropdown. Click Create function.

Task 2: Configure the Trigger

1. Click Add trigger and select EventBridge (CloudWatch Events) from the dropdown.
2. Create a new rule with Rule name: everyMinute, Rule type: Schedule expression, and Schedule expression: rate(1 minute). Click Add.

The screenshot shows the 'Basic information' tab of the AWS Lambda console. It includes fields for 'Function name' (myStopinator), 'Runtime' (Python 3.11), 'Architecture' (x86_64), and 'Permissions'. The 'Change default execution role' section is expanded, showing the 'Execution role' dropdown set to 'myStopinatorRole'.

The screenshot shows the 'Trigger configuration' tab of the AWS EventBridge console. It includes a 'Rule' section with 'Create a new rule' selected, a 'Rule name' field set to 'everyMinute', and a 'Rule type' section with 'Schedule expression' selected.

The screenshot shows the 'Rule type' section of the AWS EventBridge console. It includes a 'Schedule expression' field set to 'rate(1 minute)' and a 'Schedule expression' section with 'rate(1 minute)' entered.

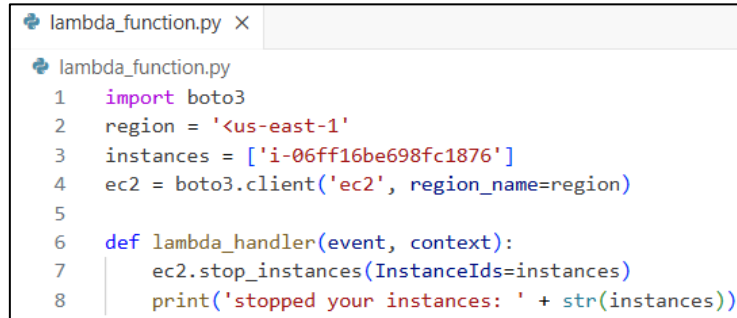
Task 3: Configure the Lambda Function

1. In the Code tab, open `lambda_function.py` and replace the existing code with the following:

```
import boto3

region = '<REPLACE_WITH_REGION>'
instances = ['<REPLACE_WITH_INSTANCE_ID>']
ec2 = boto3.client('ec2', region_name=region)

def lambda_handler(event, context):
    ec2.stop_instances(InstanceIds=instances)
    print('stopped your instances: ' + str(instances))
```



2. Replace the region placeholder with your actual AWS region code (e.g., `us-east-1`), keeping the single quotation marks.
3. Navigate to EC2, copy the instance ID of the running instance named `instance1`, and replace the instance ID placeholder in the code, keeping the single quotation marks.
4. Save the file and click Deploy. Then click the Monitor tab to view invocation count and success rate charts.

Task 4: Verify that the Lambda Function Worked

1. Return to the EC2 console and verify that `instance1` has been stopped by the Lambda function.

☐	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public
☐	Bastion Host	i-02ba135af6f02701b	Running	t2.micro	2/2 checks passed	View alarms	us-east-1a	ec2-3-235-13-7.comput...	3.235.1
☐	instance1	i-06ff16be698fc1876	Stopped	t2.micro	–	View alarms	us-east-1c	–	–

2. Try starting the instance again. Since the EventBridge trigger fires every minute, the Lambda function will stop the instance again automatically.

Conclusion: Thus, we have successfully developed and executed a serverless function using AWS Lambda in the AWS cloud environment.